IMDb Sentiment Analysis: Reproducing and Improving a CS229 Report

UE24CS352A Machine Learning mini-project, problem statement 10 · Arjun G Kanagal, Vedaraj H R

1. Problem statement

We were asked to reproduce and improve on *"Machine Learning based classification for Sentimental analysis of IMDb reviews"* by C.-L. Wu and S.-L. Shin (Stanford CS229). Sentiment analysis is the task of deciding whether a text expresses a positive or a negative opinion; here there are exactly two classes and no neutral one. The report classifies IMDb movie reviews as positive or negative. It cleans the text, turns each review into a vector of 3-gram features in three ways (binary, word count, tf-idf) and compares six classifiers: Logistic Regression, SVM, Naive Bayes, Random Forest, Boosting and a five-layer neural network. Its best accuracy is 90.6% (neural network on binary features).

Our goals were to (1) reproduce the report's Table 2, (2) check its methodology, and (3) beat 90.6% on the same test set.

2. Dataset

We use the Large Movie Review Dataset v1.0 (Maas et al., 2011), the same data as the report: 50,000 labelled IMDb reviews, split by its authors into 25,000 for training and 25,000 for testing, each half 50% positive. A review is negative if its rating is 4 or lower and positive if it is 7 or higher; ratings of 5 and 6 are excluded. Ratings are concentrated at the extremes: 1 and 10 make up about 40% of all reviews.

Reviews average 230 words (median 173, longest 2,470). No movie has more than 30 reviews. The training set covers 3,456 movies and the test set 3,581, with only one movie in common, so a model is always tested on movies it has not seen. The dataset identifies each movie only by its IMDb ID; it holds no titles or genres.

3. Approach

Reproduction. We follow the report's pipeline. Text is lowercased; HTML line breaks, punctuation, numbers and stop words are removed; each word is reduced to its root with the WordNet lemmatiser. All unigrams, bigrams and trigrams of the training reviews form one vocabulary, from which the binary, word-count and tf-idf matrices are derived, so the three representations differ only in the values they hold. Each model uses the settings listed in the report.

The report leaves three details open, and its code is no longer online:

- *Stop words.* The list is not named. The report's examples ("a", "the", "of") come from the five-word list of the tutorial it cites for its method, so we use that list, and test NLTK's full 198-word list separately.
- *"3 grams".* We read this as 1- to 3-grams together, as in the same tutorial.
- *Choosing C.* The report says only "choose the best performance". We choose C on a validation split and never on the test set.

Evaluation protocol. Hyperparameters are chosen on 20% of the training set, held out *by movie*: because the test set contains only unseen movies, a random split, with reviews of the same movie on both sides, rewards memorising movie-specific words. On a random split Naive Bayes scores 89.5%, above its own test accuracy of 88.0%; grouped by movie it scores 86.6%. The chosen model is refitted on all 25,000 training reviews and scored once on the 25,000 test reviews.

Improvements. We changed one thing at a time and kept what helped on validation data:

1. *Cleaning ablation*: the report's cleaning against NLTK's full stop-word list, against lighter cleaning that keeps stop words, negations ("not", "no") and numbers ("10/10"), and against lighter cleaning that also keeps "!", "?" and one-letter words as tokens.
2. *NB-SVM / NB-LR* (Wang and Manning, 2012): a linear classifier trained on binary n-grams scaled by their Naive Bayes log-count ratios, which combines two of the report's own models.
3. *A wider C grid*: the report's grid stops at C = 1, which over-regularises tf-idf features whose values are much smaller than counts.
4. *Pretrained transformers*: every model above reads a review as a bag of n-grams. We fine-tune DistilBERT (66 M parameters) and RoBERTa-base (125 M), which read the words in order and start from a language model pretrained on general English text, on the 80% training split for up to two epochs, keeping the epoch with the best validation accuracy. Reviews over 512 tokens keep their first 128 and last 382 tokens (Sun et al., 2019). They are not refitted on the full training set, so they see 20% less data.

4. Implementation overview

The code is a small Python package (scikit-learn, NLTK, pandas, matplotlib). `data.py` downloads the dataset and reads it into CSV files together with the movie each review is about; `preprocess.py` implements three cleaning modes and caches the cleaned text; `features.py` counts all 1- to 3-grams once and derives the binary, word-count and tf-idf matrices from the same vocabulary. `experiments/runner.py` holds the one loop every experiment uses: choose hyperparameters on a validation split grouped by movie, refit on the full training set, and score once on the test set. Each experiment script writes its own CSV with the same columns to `results/`: `linear_models.py` (Logistic Regression, SVM, Naive Bayes),

`nonlinear_models.py` (Random Forest, Boosting and the paper's five-layer DNN, on a reduced vocabulary: n-grams found in at least 5 reviews for Random Forest and at least 50 for Boosting and the DNN, so that they finish on a laptop), `improve_features.py` (cleaning ablation, NB-SVM / NB-LR, wider C grid), `improve_ensemble.py` (a wider ReLU network and an ensemble), `transformer_models.py` (DistilBERT and RoBERTa, PyTorch and Hugging Face `transformers` on an RTX 3050 GPU) and `final_comparison.py` (ensembles of the transformers, 95% confidence intervals and significance tests). `evaluate.py` computes the metrics and the comparison with the paper's Table 2, `plots.py` draws the figures, and a Streamlit app and a Jupyter notebook demonstrate the results. Every experiment script has a `--quick` mode for a smoke test on a small subset, and all random choices use one seed (42).

5. Results

Reproduction. Our Logistic Regression, SVM and Naive Bayes accuracies are within 0.35 points of every corresponding row of the paper's Table 2 (README, first table). Random Forest (210,685 features) scores 0.862, 0.860 and 0.855 for binary, count and tf-idf, about one point above the paper's 0.852, 0.849 and 0.844. The DNN and Boosting used a smaller vocabulary (17,671 features): the binary DNN scores 0.8985 against the paper's 0.906 and binary Boosting 0.8151 against 0.831. On count and tf-idf features the DNN scores 0.8953 and 0.8989 (paper: 0.898 and 0.901) and Boosting 0.8169 and 0.8165 (paper: 0.834 and 0.825). In an earlier run with the larger 210,685-feature vocabulary the binary DNN reached 0.9057, the paper's value, but it took about ten minutes, so the vocabulary cut explains most of the gap.

Improvements. The paper's best accuracy is 0.906. All numbers are on the 25,000 test reviews; nothing was chosen on them.

Model	Features	Accuracy	vs paper's best
RoBERTa-base, fine-tuned	raw text, subword tokens	0.9586	+0.0526
DistilBERT + RoBERTa ensemble	raw text, subword tokens	0.9567	+0.0507
DistilBERT, fine-tuned	raw text, subword tokens	0.9381	+0.0321
NB-LR	binary, minimal cleaning	0.9160	+0.0100
NB-SVM	binary, minimal cleaning	0.9157	+0.0097
SVM, wider C grid	tf-idf	0.9089	+0.0029
MLP (one ReLU layer, 128 units)	binary, 23,700 features	0.8931	-0.0129
Ensemble: NB-LR + SVM (tf-idf)	mixed	0.9182	+0.0122

The transformers give by far the largest gain: RoBERTa reaches 95.9% (95% confidence interval 95.6 to 96.1%), 5.3 points above the paper. A two-proportion z-test puts every model from NB-LR upwards significantly above 0.906 ($p < 0.0001$), but not the wider-C SVM ($p = 0.13$). An exact McNemar test on the same test reviews separates RoBERTa from NB-LR ($p = 6e-140$). Validation chose the DistilBERT + RoBERTa ensemble, which scored 0.2 points below RoBERTa alone on the test set; the difference is within noise. Among the n-gram models, Naive Bayes scaling of the features is the largest single gain (about 1.3 points over the same linear models); keeping stop words and negations is worth 1.3 to 1.7 points compared with removing NLTK's full stop-word list. The ensemble averages the scores of NB-LR and the tf-idf SVM, the pair with the best validation accuracy of the 11 combinations we tried (0.9070); it has the best test accuracy (0.9182) and ROC-AUC (0.9739, against 0.9723 for NB-LR alone). Our wider MLP (0.8931) did not beat the paper-style DNN (0.8985 on a vocabulary of similar size), so the network design was not what improved the results; note that the two use different cleaning, so this is not a controlled comparison.

6. Conclusions

We reproduced all 18 rows of the paper's Table 2 (the linear models within 0.35 points, the others within 1.7) and beat its best accuracy of 90.6% in two ways. Keeping n-gram features, NB-LR reaches 91.6% and an ensemble of NB-LR and a tf-idf SVM 91.8%, from Naive Bayes log-count ratios, lighter cleaning that keeps negations and a wider search over the regularisation strength. Replacing n-grams with a pretrained transformer that reads word order, fine-tuned RoBERTa-base reaches 95.9%, cutting the paper's error rate from 9.4% to 4.1%; both gains are statistically significant. A wider feed-forward network on n-grams did not help. Along the way we found problems in the paper: its "precision" columns behave like recall, two rows of Table 2 are internally inconsistent, and the SVM uses values of C outside the grid it reports. **Limitations:** every number comes from one train/test split and one seed. With 25,000 test reviews the standard error of an accuracy is about 0.17 points, so the 0.2-point differences between our ensembles and their best members are within noise. The transformers need a GPU and only the number of epochs was tuned. The paper's code is not available, so some details are our assumptions, and the DNN and Boosting used a reduced vocabulary. **Future work:** larger transformers (RoBERTa-large), a third epoch, and results averaged over several seeds.